Course: CSC3105 - Data Analysis

Title: Mini Project Report

Group: 11

Group Members:

Name	GUID	SIT ID	Contributions
Swathi Ravichandran	3070703R	2402695	Classification
Borse Srusthi Ganesh	3070662B	2401618	Regression
Subhashini Rajkumar	3070545R	2402769	Cleaning of data prep
Lai Yueyin Shyann	3070689L	2402456	PCA on data prep
Foo Guo Wei	3070704F	2402716	Visualization

Overleaf Report Link: https://www.overleaf.com/project/69d13ddb0455360bd9838503
Youtube Demo Video Link: https://youtu.be/wUZeijmQXrQ

Problem Definition

Warehouses are increasingly using autonomous robots to navigate the space, to automate the restocking, auditing and collection activities. These warehouses are large, often exceeding 100,000–500,000 sq ft, and are filled with dense shelving units. This is a LOS/NLOS environment, where signals can travel through a direct path (LOS) or an indirect path (NLOS) through metal shelves. To capture these signals, the robot's navigation systems rely on Ultra-Wideband (UWB) anchors placed at fixed points to determine their exact coordinates.

However, standard UWB usually fails when a robot moves behind a metal shelving unit, which may cause routing problems. Robots are more likely to crash fatally into shelves or fail to perform correctly because of this. This may cause disruptions to inventory, leading to financial losses. This highlights the importance of robust and reliable navigation systems for autonomous warehouse operations.

Problem Analysis

To ensure autonomous robots perform accurate indoor positioning in warehouses with both direct and indirect paths, we propose a hybrid two-path UWB ranging solution. In conventional UWB systems, NLOS signals are treated as noise, causing overestimated distances and incorrect positioning. This leads to poor path planning and potential collisions.

Instead of treating the signal as a simple one-path problem, our solution explicitly models multipath behaviour by extracting the second dominant peak from the Channel Impulse Response (CIR) after the first-path index (FP_IDX). From this, we derive new features such as the second peak position and second peak amplitude, and use them to estimate a reflected-path distance in addition to the normal direct-path distance. These engineered features are then combined with PCA-reduced CIR features and standard radio features, and used in machine learning models for both classification and regression.

The originality of our solution lies in the way we combine signal processing and data analytics. Rather than only classifying LOS/NLOS or only predicting one distance value, our approach integrates: 1) CIR peak extraction, 2) feature engineering for multipath, 3) dimensionality reduction on high-dimensional CIR data, and 4) a multi-output regression framework that predicts both direct and reflected path distances. In the context of this project, this is original because it goes beyond a standard baseline model and directly addresses the real source of UWB ranging error, which is multipath propagation in NLOS conditions.

This is how we break down our problem:

1. We first break down the problem into two main tasks: identifying whether the signal is LOS or NLOS, and estimating the distance accurately even when reflections are present.
2. We preprocess the data by combining all dataset parts, checking for missing values and outliers, and normalising the features. We also preserve FP_IDX because it is important for later signal analysis.

3. We engineer features from the CIR signal by detecting the second dominant peak using peak-finding. This implementation is guided by the physics rule that dictates that UWB signals in indoor environments consist of a direct path (Path 1) and a primary reflection (Path 2). This allows us to capture reflected-path, PEAK2_POS and PEAK2_AMP.
4. Since the CIR data is very high-dimensional, we apply PCA to reduce dimensionality while preserving about 95% of the variance.
5. We then use two analytics models:
    - A classifier to determine the propagation condition [state 0: LOS/NLOS or state 1: NLOS/NLOS]
    - A multi-output regressor to predict both Path 1 (direct distance) and Path 2 (reflected distance).
6. Finally, we evaluate the model on unseen test data. Results are discussed in full in the Plots and Results section.

Algorithm

1. Data Preparation Algorithms

Feature Engineering

We performed feature engineering to capture the multipath propagation effects present in NLOS environments. The Channel Impulse Response (CIR), which represents signal strength over time, is analysed to identify dominant signal paths. While the first path corresponds to the direct signal (if present), additional peaks in the CIR represent reflected signals caused by obstacles.

To capture this behaviour, a signal processing approach is applied to extract the second dominant peak from the CIR. For each sample, the algorithm starts from the first path index (FP_IDX) and examines the remaining signal. Peak detection is performed using a peak-finding algorithm to identify local maxima in the CIR. The peak with the highest amplitude after the first path is selected as the second dominant path.

From this, two key features are engineered:

Second peak position: represents the time delay of the reflected signal
Second peak amplitude: represents the strength of the reflected signal

If no clear second peak is detected, fallback values are assigned to maintain consistency in the dataset.

Data Cleaning and Feature Scaling

Before feature engineering, all 7 dataset parts were combined into one full dataset. Missing values were checked and summarised, but no imputation was performed.

Similarly, outliers were identified using the Interquartile Range (IQR) rule, but they were only reported and not removed or capped.

Feature scaling was performed using z-score normalization: z = x - μ/σ.
In the first preprocessing stage, all feature columns were scaled except `FP_IDX`, `NLOS`, and `RANGE`. `FP_IDX`.

Afterwards, the dataset was split into training and testing sets using an 80:20 stratified train-test split based on the `NLOS` label. The dataset contains 42,000 samples in total, with 21,000 LOS and 21,000 NLOS observations, resulting in 33,600 training samples and 8,400 testing samples.

PCA was applied to these CIR features using the training data, with `n_components = 0.95`. This reduces 1,017 CIR features to 860 principal components.

These PCA features were then combined with 14 structured features, giving a final input of 860 features:

- 12 non-CIR radio features (`FP_IDX`, `FP_AMP1`, `FP_AMP2`, `FP_AMP3`, `STDEV_NOISE`, `MAX_NOISE`, `RXPACC`, `CH`, `FRAME_LEN`, `PREAM_LEN`, `BITRATE`, `PRFR`)
- 2 engineered features (`PEAK2_POS` and `PEAK2_AMP`)

The final outputs of the data preparation stage are:

- `X_train.npy` and `X_test.npy`: final feature matrices for training and testing
- `y_train_class.npy` and `y_test_class.npy`: classification targets for LOS/NLOS prediction
- `y_train_reg.npy` and `y_test_reg.npy`: regression targets for Path 1 and Path 2 distance estimation
- `metadata.json`: PCA configuration, explained variance information, and class distribution details

2. Data Mining Algorithms

Random Forest Classifier

The Random Forest Classifier acts as a “Gatekeeper” that determines the propagation condition: State 0 (LOS-NLOS) or State 1 (NLOS-NLOS). Using a classifier also allows for error mitigation. Some examples of errors are when a signal is NLOS but takes longer to arrive making the distance seem longer than it is or if an NLOS is treated as a LOS signal the navigation results will be vastly different. The classifier flags the “obstructed” signals so that the system can apply a correction or ignore the corrupted data.

We chose the random forest for its ability to handle non-linear signal behaviour. By studying the dataset, we found that the relationship between CIR features and actual distance is not linear, especially in multipath environments. Random forest can accurately capture non-linear mappings using decision trees. Instead it builds a “forest” of many individual decision trees and merges their results together to get a more accurate and stable prediction as it does not rely on just one decision-making model. Random Forest is also able to handle noise in the dataset, as UWB data is highly noisy due to thermal interference.

Input:
The model processes a multidimensional feature vector derived from the Channel Impulse Response (CIR).

- **Physical Signal Features:** Key metrics include First Path Index (*FP_IDX*), First Path Amplitude (*FP_AMP*) and received Preamble Accumulation (*RXPACC*).
- **Noise & Peak Analysis:** Standard Deviation of Noise (*STDEV_NOISE*) and the amplitude/position of the second peak (*PEAK2_AMP, PEAK2_POS*).
- **Dimensionality Reduction:** The model utilises PCA (Principal Component Analysis) components (*PCA_0 - PCA_11*) to capture complex signal patterns that raw metrics might miss

Model Structure:
The model utilizes a Random Forest Classifier consisting of an ensemble of 300 individual decision trees. Rather than relying on a single complex model, this ensemble learning approach uses a "forest" of simpler trees to reach a collective consensus. Each tree is trained using bootstrapping on a random subset of the data, a process that ensures the model remains robust and does not overfit to specific signal noise. Additionally, the structure incorporates a "*balanced*" class weighting to ensure the algorithm pays equal attention to both LOS and NLOS states, effectively mitigating any potential bias caused by differences in the sample distribution.

Training Process:
Supervised learning was selected because the project objective is to map specific signal characteristics to known environmental "ground truths," specifically State 0 (LOS-NLOS) and State 1 (NLOS-NLOS). This approach is essential for categorical mapping, providing the labeled training data required to learn the complex mathematical boundaries between distinct physical states. Furthermore, it enables rigorous performance benchmarking through objective metrics, such as precision, recall, and accuracy, to verify that the "Gatekeeper" reliably identifies obstructions before data reaches the localization engine. Finally, this framework effectively handles signal complexity by training the model to recognize specific "fingerprints" of multipath interference within a high-dimensional dataset of raw hardware metrics and PCA-compressed components.

To ensure the model can generalize to unseen signal environments rather than simply memorizing the training set, an **80:20 train/test split** was implemented. This provides a substantial evaluation set of 8,400 samples to verify the model’s

real-world reliability. The "forest" consists of 300 individual decision trees with a maximum depth of 25. This depth allows the model to perform complex logical splits, essential for distinguishing subtle signal reflections, without introducing excessive computational complexity or overfitting. At every decision split, the model is restricted to looking at a random subset of features (*max_features='log2'*). This forces individual trees to remain diverse and uncorrelated, ensuring that the final "consensus" prediction is not overly reliant on a single metric like RXPACC or a specific PCA component. The pipeline incorporates *class_weight='balanced'*. This ensures the algorithm treats State 0 and State 1 with equal importance during the learning phase, which is reflected in the balanced Macro and Weighted averages of 0.85 in the final performance report.

Prediction Process:
When a new signal enters the system, the prediction process follows a structured voting logic starting with feature mapping, where the new Channel Impulse Response (CIR) data is transformed into the exact same feature set including PCA components used during the initial training phase. This is followed by parallel inference, where each of the 300 decision trees in the forest independently analyses these features to cast an individual "vote" for either State 0 (LOS-NLOS) or State 1 (NLOS-NLOS). Finally, the model performs probability thresholding by calculating the mean probability across the entire ensemble; if the majority of these votes favor a specific state, that consensus becomes the final classification for the signal.

Output:
The algorithm produces 2 primary outputs used for downstream tasks.

- **Predicted Class:** A discrete label identifying the two-path state (State 0 vs. State 1).
- **Class Probabilities:** An AUC score of **0.8941** demonstrates the model's high confidence in distinguishing between clear and obstructed paths.
- **Usage:** This output is used to flag NLOS conditions, allowing the system to decide whether to trust the signal or apply a bias correction.

Detailed Performance (Brief Requirement):

	precision	recall	f1-score	support
State 0 (LOS-NLOS)	0.80	0.92	0.86	4200
State 1 (NLOS-NLOS)	0.90	0.77	0.83	4200
accuracy			0.85	8400
macro avg	0.85	0.85	0.85	8400
weighted avg	0.85	0.85	0.85	8400

This table evaluates the model's predictive power using four statistical pillars:

- **Precision (Exactness):** Indicates the percentage of signals labeled as a specific state that were actually that state.
    - **State 1 (0.90):** When the model predicts an obstruction, it is correct **90%** of the time.
- **Recall (Completeness):** Indicates the percentage of all actual signals of a specific state that the model successfully caught.

    - **State 0 (0.92):** The model successfully identifies **92%** of all clear paths.
    - **State 1 (0.77):** The model only catches **77%** of actual obstructed signals, leaving a 23% gap where obstructions are missed.
- **F1-Score (Balance):** The harmonic mean of Precision and Recall. It shows that **State 0 (0.86)** has slightly more reliable overall performance than **State 1 (0.83)**.
- **Support:** The raw number of samples in each class (4,200 each), confirming a perfectly **balanced dataset** for testing.

Random Forest Regressor

Regression is used to derive the actual physical distance in meters from the CIR data and engineered multi-path features. The classification output provides contextual information to determine whether the measured distance is reliable or affected by environmental bias. When a signal is classified as NLOS, the measured first-path distance is often overestimated due to signal reflection or obstruction. To address this, the regressor incorporates both the direct-path estimate (LOS/NLOS) and reflected-path features (NLOS/NLOS) derived from the CIR analysis. By learning the relationship between these features, the model should correct for NLOS induced delay and accurately estimate distances.

As previously stated in the Classifier section, Random Forest's ability to handle noise and non-linear relationships makes it adequate for this implementation. Overall, Random Forest Regressor improves the accuracy of distance estimation, which directly enhances localization precision and enables more reliable navigation in both LOS and NLOS conditions.

Input:
The regression model takes the following features inside X_train and X_tests as input:

- CIR features, reduced using PCA
- Engineered features
    - Second peak position
    - Second peak amplitude
    - Derived reflected path information

These combined features capture both direct-path and reflected-path signal characteristics, enabling the model to learn the relationship between signal behaviour and physical distance.

Model Structure:
To handle the prediction of two dominant signal paths, a MultiOutputRegressor wrapper is used. This trains two separate Random Forest models in parallel, one for each target output: the direct path distance (Path 1) and the reflected path distance (Path 2). This structure allows the model to learn both direct and reflected signal behaviours simultaneously, which is essential for accurate distance estimation in multipath environments.

Our key model parameters include the number of trees (n_estimators = 100), maximum tree depth (max_depth = 15), and minimum samples per leaf (min_samples_leaf = 5), which control model complexity and prevent overfitting.

- N_estimators = 100. We tested the code with 200 trees and 50 trees. 100 trees was the sweet spot that provided stable and reliable predictions while maintaining reasonable training time. Any number of trees higher than this only yielded a marginal performance improvement. Meanwhile, 50 trees underperformed as there was not enough data to train the model on.
- Max_depth = 15. This allows the model to capture non-linear relationships in UWB signal features while preventing excessive overfitting to noise present in NLOS conditions.
- Min_samples_leaf = 5. Setting this to 5 ensures that predictions are based on sufficient data, reducing sensitivity to noise and outliers in the dataset.

Prediction Process:
The random forest regressor works by processing the input features through each decision tree. The sample goes through feature-based splits until reaching a leaf node, which then outputs a predicted distance value. Since a MultiOutputRegressor is used, separate predictions are generated for each target, corresponding to the direct path (Path 1) and the reflected path (Path 2).

The final predicted distance for each path is obtained by averaging the outputs of all decision trees.

Algorithm Output:
The model outputs two continuous values representing the estimated distances of the dominant signal paths: the direct path distance and the reflected path distance
These predicted distances are used for indoor localization, enabling more accurate estimation of the robot's position. In particular, the inclusion of the reflected path allows the system to account for multipath propagation effects in NLOS environments, reducing bias in distance estimation. As a result, the system improves positioning accuracy and supports more reliable navigation in complex indoor environments.

Evaluation Criteria:

Train Performance:

Metric	**Path 1 (Direct)**	**Path 2 (Reflection)**
RMSE	0.8400 m	0.8286 m
MAE	0.6313 m	0.6197 m
R^2	0.8732	0.8840

The model demonstrates strong performance during training, achieving RMSE values below 1 meter and R^2 values above 0.87 for both direct and reflected signal paths. This indicates that the model effectively learns the relationship between UWB signal features and spatial positioning.

Test Performance:

Metric	**Path 1 (Direct)**	**Path 2 (Reflection)**
RMSE	1.4065 m	1.4189 m
MAE	1.0794 m	1.0872 m
R^2	0.6402	0.6550

However, when evaluated on unseen test data, the RMSE increases to approximately 1.41 meters and R^2 drops to around 0.65, suggesting moderate generalization capability and some degree of overfitting. Despite this, the comparable performance between direct (LOS) and reflected (NLOS) paths indicates that the model is able to extract meaningful information from NLOS signals rather than discarding them.

Algorithm Pseudocode

Random Forest:
 	
INPUT: Training data (X, y)
 	
FOR each tree in forest:
  Create bootstrap sample from data
  Build decision tree:
  WHILE not stopping condition:
  Randomly select subset of features
  Find best split
  Split node
 	
STORE all trees
 	
PREDICTION:
  FOR each tree:
  predict output
  RETURN average (regression) OR majority vote (classification)
	Used in:
	train_rf_regressor.py
	
	Purpose:
	predict 2 outputs
	- (path 1 & path 2 distance)

Random Forest Classifier:
 	
INPUT: X, y (class labels)
 	
FOR each tree:
  Sample data with replacement
  Build decision tree using Gini/Entropy
 	
PREDICT:
  Each tree votes for a class
  RETURN class with most votes
	Used in:
	train_rf_classifier.py
	
	Purpose:
	predict 2 outputs
	- (path 1 & path 2 distance)

Multi-Output Regression:
 	
INPUT: X, Y (multiple outputs)
 	
FOR each output column y_i:
  Train separate model M_i using X → y_i
 	
PREDICT:
  FOR each model M_i:
  predict y_i
  COMBINE predictions into vector
	Wrapper around Random Forest
	
	Purpose:
	Trains one model per output dimension

Principal Component Analysis (PCA): INPUT: Data matrix X	Used in: phase1_data_prep_continuation.py

STANDARDIZE X COMPUTE covariance matrix C COMPUTE eigenvectors and eigenvalues of C SORT eigenvectors by descending eigenvalues SELECT top k components (e.g. 95% variance) TRANSFORM data: X_new = X × selected eigenvectors	Purpose: Reduce CIR signal dimensionality - keep 95% variance
Data Processing: INPUT: dataset D, test_size SHUFFLE dataset SPLIT: train_size = 1 - test_size D_train = first portion D_test = remaining portion RETURN D_train, D_test	Purpose: Train-Test Split - Random sampling with stratification - Split dataset into training/testing set
Signal Processing: INPUT: signal array FOR each index i: IF signal[i] > neighbors: mark as peak RETURN all peak indices and heights	Purpose: Peak Detection - Used to detect second signal path - Selects highest peak
Custom Feature Extraction: INPUT: dataframe df, CIR columns FOR each row i: start_idx = FP_IDX[i] signal = CIR[i][start_idx : end] peaks = find_peaks(signal) IF peaks exist: find peak with highest amplitude peak2_pos = peak index + start_idx	Purpose: - Slice signal after first path - Find and select strongest peaks - Compute derived features

peak2_amp = peak height ELSE: peak2_pos = start_idx peak2_amp = 0 RETURN arrays of peak2_pos and peak2_amp	
Root Mean Square Error: RMSE = sqrt(mean((y_true - y_pred)^2))	Purpose: Penalise large error - Used to evaluate distance prediction accuracy - Lower RMSE = better model
Mean Absolute Error: MAE = mean(|y_true - y_pred|)	Purpose: Average error magnitude - Show average error in meters
R^2 Score: $R^2 = 1 - (SS_{res} / SS_{total})$	Purpose: Compare vs Baseline - Shows how well the model captures signal to distance relationship
Confusion Matrix: FOR each prediction: count TP, TN, FP, FN RETURN matrix: [[TN, FP], [FN, TP]]	Purpose: Understand Mistakes - Breaks down classification performance into detailed outcome TP (True Positive): correct NLOS TN (True Negative): correct LOS FP (False Positive): False NLOS FN (False Negative): False LOS
ROC Curve: FOR different thresholds: compute TPR and FPR PLOT FPR vs TPR	Purpose: Evaluate Classifiers - model separation between classes across all thresholds - LOS vs NLOS (TP vs FP rates)
Area Under Curve: AUC = area under ROC curve	Purpose: Compare Models - Gives threshold-independent performance score

Source Codes

phase1_data_prep.py

```python
from pathlib import Path
import numpy as np
import pandas as pd

DATASET_GLOB = "uwb_dataset_part*.csv"
TARGET_COLS = ["NLOS", "RANGE"]
# IMPORTANT: Keep FP_IDX raw so we can use it for signal processing in 1.2
DONT_SCALE = ["FP_IDX"] + TARGET_COLS

def load_and_combine(dataset_dir: Path) -> pd.DataFrame:
    files = sorted(dataset_dir.glob(DATASET_GLOB))
    if not files:
        raise FileNotFoundError(f"No files found with pattern {DATASET_GLOB}")
    frames = [pd.read_csv(file) for file in files]
    return pd.concat(frames, ignore_index=True)

def summarize_missing_values(df: pd.DataFrame):
    return df.isna().sum(), int(df.isna().sum().sum())

def summarize_outliers_iqr(df: pd.DataFrame, feature_cols: list[str]):
    q1, q3 = df[feature_cols].quantile(0.25), df[feature_cols].quantile(0.75)
    iqr = q3 - q1
    outlier_mask = (df[feature_cols] < (q1 - 1.5 * iqr)) | (df[feature_cols] > (q3 + 1.5 * iqr))
    return outlier_mask.sum(), int(outlier_mask.sum().sum())

def zscore_scale_features(df: pd.DataFrame, feature_cols: list[str]) -> pd.DataFrame:
    # Only scale features that aren't in our DONT_SCALE list
    to_scale = [c for c in feature_cols if c not in DONT_SCALE]
    means = df[to_scale].mean()
    stds = df[to_scale].std(ddof=0).replace(0, 1.0)
    df[to_scale] = ((df[to_scale] - means) / stds).astype(np.float32)
    return df
```

```python
def main() -> None:
    project_root = Path(__file__).resolve().parent
    dataset_dir = project_root / "dataset"
    output_dir = project_root / "processed"
    output_dir.mkdir(exist_ok=True)

    df = load_and_combine(dataset_dir)
    _, missing_total = summarize_missing_values(df)

    feature_cols = [c for c in df.columns if c not in TARGET_COLS]
    _, outlier_total = summarize_outliers_iqr(df, feature_cols)

    # Scaling Step (Skips FP_IDX)
    df = zscore_scale_features(df, feature_cols)

    combined_csv_path = output_dir / "uwb_combined_scaled.csv"
    df.to_csv(combined_csv_path, index=False)
    np.save(output_dir / "uwb_combined_scaled.npy", df.to_numpy(dtype=np.float32))
    print("Phase 1.1 Completed: FP_IDX preserved for processing.")

if __name__ == "__main__":
    main()
```

For the phase1_data_prep stage, the dataset files were first loaded and merged into one combined dataset. Basic data quality checks were then performed by summarizing missing values and detecting possible outliers using the IQR method. After that, the input features

were standardized using z-score scaling to make them comparable, while FP_IDX and the target columns (NLOS and RANGE) were kept in their original form for later processing. Finally, the prepared dataset was saved in both CSV and NumPy formats for the next phase.

phase1_data_prep_continuation.py

```python
import json
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from scipy.signal import find_peaks

RANDOM_STATE = 42
TEST_SIZE = 0.2

def extract_second_peak_features(df, cir_cols):
    """Signal processing to find the 2nd dominant path."""
    peak2_pos = []
    peak2_amp = []
    cir_matrix = df[cir_cols].values
    fp_indices = df['FP_IDX'].values

    for i in range(len(df)):
        start_idx = int(fp_indices[i])
        # Look at the signal AFTER the first path
        signal = cir_matrix[i, start_idx:]
        peaks, props = find_peaks(signal, height=0)

        if len(peaks) > 0:
            highest_peak_idx = np.argmax(props['peak_heights'])
            peak2_pos.append(peaks[highest_peak_idx] + start_idx)
            peak2_amp.append(props['peak_heights'][highest_peak_idx])
        else:
            # If no clear 2nd peak, use first path info as a baseline
            peak2_pos.append(start_idx)
            peak2_amp.append(0)

    return np.array(peak2_pos), np.array(peak2_amp)
```

For the continuation of the phase 1 preparation stage, the function to extract the second-peak is made. And snippets of the main function were shared: It is a walkthrough of how classifications were made, data was split to train-test subsets, before scaling and PCA reduction was performed. Finally, though not shown, the dataset produced plots and was saved as NumPy formats for the next phase.

```python
def main():
    project_root = Path(__file__).resolve().parent
    data_path = project_root / "processed" / "uwb_combined_scaled.csv"
    output_dir = project_root / "data_prep_output"
    plot_dir = project_root / "training" / "plots"
    output_dir.mkdir(exist_ok=True)
    plot_dir.mkdir(parents=True, exist_ok=True)


    df = pd.read_csv(data_path)
    save_feature_boxplots(df, plot_dir / "feature_boxplots_by_class.png")

    # --- 1. TARGET GENERATION (The "Two Path" Fix) ---
    # Path 1 Distance (Standard)
    y_reg_p1 = df["RANGE"].values

    # Path 2 Distance Estimation (Based on Peak Position)
    # Hint: Distance diff = (Peak Index Diff) * (Speed of light * Sample Time)
    # 0.00468m is a common index-to-distance conversion for UWB CIR
    cir_cols = [c for c in df.columns if c.startswith("CIR")]
    p2_pos, p2_amp = extract_second_peak_features(df, cir_cols)

    index_diff = p2_pos - df['FP_IDX'].values
    y_reg_p2 = y_reg_p1 + (index_diff * 0.00468) # Estimating Ground Truth for Path 2

    # Combine into a Multi-Output Target [N, 2]
    y_reg_combined = np.column_stack([y_reg_p1, y_reg_p2])

    # Classification: Applying the "Golden Rule"
    # State 0 (LOS-NLOS): If first path is LOS (0)
    # State 1 (NLOS-NLOS): If first path is NLOS (1)
    y_joint_class = df["NLOS"].values

    # --- 2. SPLIT DATA ---
    X = df.drop(columns=["NLOS", "RANGE"])
    X_train, X_test, y_train_class, y_test_class, y_train_reg, y_test_reg = train_test_split(
        X, y_joint_class, y_reg_combined,
        test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y_joint_class
    )
```

```python
    # --- 3. FEATURE ENGINEERING ---
    print("Engineering features for Path 2...")
    # Re-extract for split sets
    X_train_p2_pos, X_train_p2_amp = extract_second_peak_features(X_train, cir_cols)
    X_test_p2_pos, X_test_p2_amp = extract_second_peak_features(X_test, cir_cols)

    non_cir_cols = [c for c in X.columns if not c.startswith("CIR")]
    X_train_struct = np.column_stack([X_train[non_cir_cols], X_train_p2_pos, X_train_p2_amp])
    X_test_struct = np.column_stack([X_test[non_cir_cols], X_test_p2_pos, X_test_p2_amp])

    # Scaling
    scaler_non_cir = StandardScaler()
    X_train_non_cir_scaled = scaler_non_cir.fit_transform(X_train_struct)
    X_test_non_cir_scaled = scaler_non_cir.transform(X_test_struct)

    # PCA on CIR
    scaler_cir = StandardScaler()
    X_train_cir_scaled = scaler_cir.fit_transform(X_train[cir_cols])
    X_test_cir_scaled = scaler_cir.transform(X_test[cir_cols])

    # PCA chart call
    pca = PCA(n_components=0.95, random_state=RANDOM_STATE)
    X_train_cir_pca = pca.fit_transform(X_train_cir_scaled)
    X_test_cir_pca = pca.transform(X_test_cir_scaled)
    save_pca_explained_variance_plot(pca, plot_dir / "pca_explained_variance.png")

    # Final Stacking
    X_train_final = np.hstack([X_train_non_cir_scaled, X_train_cir_pca])
    X_test_final = np.hstack([X_test_non_cir_scaled, X_test_cir_pca])

```

Source code files train_rf_classifier.py and train_rf_regressor.py are shown. All major steps, library imports, variables and functions are commented within the source file. Classifier and Regressor codes are explained in their respective sections above.

train_rf_classifier.py

```python
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from pathlib import Path
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    confusion_matrix, classification_report,
    roc_curve, auc, accuracy_score, roc_auc_score
)

def main():
    # 1. Setup paths
    root = Path(__file__).resolve().parent.parent
    data_dir = root / "data_prep_output"
    plot_dir = root / "training" / "plots"
    plot_dir.mkdir(parents=True, exist_ok=True)

    # 2. Load the 80/20 split data
    try:
        X_train = np.load(data_dir / "X_train.npy")
        y_train = np.load(data_dir / "y_train_class.npy")
        X_test = np.load(data_dir / "X_test.npy")
        y_test = np.load(data_dir / "y_test_class.npy")
    except FileNotFoundError:
        print("Error: Files not found. Run your prep scripts with 80/20 split first!")
        return

    print(f"--- Training Joint-Class Gatekeeper (80/20 Split) ---")
```

```python
# 3. Classifier Settings:
# max_features=None ensures the model looks at PEAK2_AMP at every split.
# class_weight='balanced' handles the LOS/NLOS sample distribution.
clf = RandomForestClassifier(
    n_estimators=300,
    max_depth=25,
    max_features='log2',
    class_weight='balanced',
    random_state=42,
    n_jobs=-1
)
clf.fit(X_train, y_train)

# 4. Generate Predictions
y_pred = clf.predict(X_test)
y_prob = clf.predict_proba(X_test)[:, 1]

# 5. Terminal Report (Joint-Class Specification)
target_names = ['State 0 (LOS-NLOS)', 'State 1 (NLOS-NLOS)']

print("\n" + "="*50)
print("      TERMINAL REPORT: TWO-PATH CLASSIFICATION")
print("="*50)

acc = accuracy_score(y_test, y_pred)
auc_val = roc_auc_score(y_test, y_prob)
cm = confusion_matrix(y_test, y_pred)

print(f"Accuracy: {acc:.4f}")
print(f"ROC AUC:  {auc_val:.4f}")

print("\nConfusion Matrix Breakdown:")
print(f"True State 0 (Correct):  {cm[0][0]}")
print(f"False State 1 (Mistake): {cm[0][1]}")
print(f"False State 0 (Mistake): {cm[1][0]} <-- False LOS (Critical Weakness)")
print(f"True State 1 (Correct):  {cm[1][1]}")
```

```python
print("\nDetailed Performance (Brief Requirement):")
print(classification_report(y_test, y_pred, target_names=target_names))

# Feature Importance Terminal Print (Proving Signal Processing)
importances = clf.feature_importances_
indices = np.argsort(importances)[::-1]

# Mapping indices based on stacking in data_prep_continuation:
# 0: FP_IDX, 1: FP_AMP, 2: RXPACC, 3: STDEV_NOISE, 4: PEAK2_POS, 5: PEAK2_AMP, 6+: PCA
feature_map = {
    0: "FP_IDX",
    1: "FP_AMP",
    2: "RXPACC",
    3: "STDEV_NOISE",
    4: "PEAK2_POS",
    5: "PEAK2_AMP"
}

print("\nTop 10 Feature Importance Ranking:")
for i in range(10):
    idx = indices[i]
    name = feature_map.get(idx, f"PCA_Comp_{idx-6}")
    print(f"{i+1:2}. {name:20} : {importances[idx]:.4f}")

# 6. Plot Generation
print("\n--- Generating Visualization Files ---")

# Chart 1: Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=target_names, yticklabels=target_names)
plt.title("Joint-Class Confusion Matrix: Two-Path States")
plt.ylabel('Actual State')
plt.xlabel('Predicted State')
plt.savefig(plot_dir / "confusion_matrix.png")
plt.close()
```

```python
# Chart 2: Feature Importance
top_n = 10
top_indices = indices[:top_n]
top_names = [feature_map.get(i, f"PCA_{i-6}") for i in top_indices]
# assign colors: blue for signal features (idx < 6), orange for PCA components (idx >= 6)
colors = ["#2196F3" if i < 6 else "#FF7043" for i in top_indices]
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111)
bars = ax.barh(y=range(top_n), width=importances[top_indices][::-1], color=colors[::-1], edgecolor="white", linewidth=0.6, height=0.65)
# x y axis lable
ax.set_yticks(range(top_n))
ax.set_yticklabels(top_names[::-1], fontsize=11)
ax.set_xlabel("Mean Decrease in Impurity", fontsize=11)
ax.set_title(
    f"Top {top_n} Feature Importance (Gatekeeper)\n"
    f"(Accuracy: {acc:.4f}  |  ROC AUC: {auc_val:.4f})",
    fontsize=13, fontweight="bold", pad=5
)
 # Value annotations for each bar
for bar, imp in zip(bars, importances[top_indices][::-1]):
    ax.text(
        bar.get_width() + 0.0005,
        bar.get_y() + bar.get_height() / 2,
        f"{imp:.4f}",
        va="center", ha="left", fontsize=9, color="#333333"
    )
# Legend
legend_handles = [
    mpatches.Patch(color="#2196F3", label="Signal features"),
    mpatches.Patch(color="#FF7043", label="PCA components"),
]
ax.legend(handles=legend_handles, loc="lower right", fontsize=10, framealpha=0.9)
```

```python
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.set_xlim(0, importances[top_indices].max() * 1.18)
plt.tight_layout()
plt.savefig(plot_dir / "(v2)feature_importance.png")
plt.close()

# Chart 3: ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_prob)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {auc_val:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC)')
plt.legend(loc="lower right")
plt.savefig(plot_dir / "roc_curve.png")
plt.close()

# 7. Final Save for Regressor
np.save(data_dir / "y_test_pred_class.npy", y_pred)
print(f"\n[SUCCESS] Terminal results printed and plots saved to: {plot_dir}")

__name__ == "__main__":
main()
```

Train_rf_regressor.py

```python
import numpy as np
import matplotlib.pyplot as plt
import joblib
from pathlib import Path
from sklearn.ensemble import RandomForestRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score


def evaluate(y_true, y_pred, path_name):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae  = mean_absolute_error(y_true, y_pred)
    r2   = r2_score(y_true, y_pred)
    print(f"\n  Results for {path_name}:")
    print(f"  RMSE : {rmse:.4f} m")
    print(f"  MAE  : {mae:.4f} m")
    print(f"  R²   : {r2:.4f}")
    return rmse, mae, r2


def plot_two_paths(y_test, y_pred, plot_dir):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    # Path 1
    rmse1 = np.sqrt(mean_squared_error(y_test[:, 0], y_pred[:, 0]))
    mae1  = mean_absolute_error(y_test[:, 0], y_pred[:, 0])
    r2_1  = r2_score(y_test[:, 0], y_pred[:, 0])

    ax1.scatter(y_test[:, 0], y_pred[:, 0], alpha=0.3, color='blue', s=5)
    lims1 = [min(y_test[:, 0].min(), y_pred[:, 0].min()),
             max(y_test[:, 0].max(), y_pred[:, 0].max())]
    ax1.plot(lims1, lims1, 'r--', lw=2, label='Perfect prediction')
```

training > train_rf_regressor.py > main

```python
def plot_two_paths(y_test, y_pred, plot_dir):
    ax1.set_title("Path 1 (Direct): Actual vs Predicted")
    ax1.set_xlabel("Actual Range (m)")
    ax1.set_ylabel("Predicted Range (m)")
    ax1.legend()
    # add metrics box
    ax1.text(0.04, 0.97, f"RMSE : {rmse1:.4f} m\nMAE  : {mae1:.4f} m\nR²     : {r2_1:.4f}",
             transform=ax1.transAxes, fontsize=10, verticalalignment='top', fontfamily='monospace',
             bbox=dict(boxstyle='round,pad=0.5', facecolor='lightyellow', edgecolor='#CCCCCC', alpha=0.9))

    # Path 2
    rmse2 = np.sqrt(mean_squared_error(y_test[:, 1], y_pred[:, 1]))
    mae2  = mean_absolute_error(y_test[:, 1], y_pred[:, 1])
    r2_2  = r2_score(y_test[:, 1], y_pred[:, 1])

    ax2.scatter(y_test[:, 1], y_pred[:, 1], alpha=0.3, color='green', s=5)
    lims2 = [min(y_test[:, 1].min(), y_pred[:, 1].min()),
             max(y_test[:, 1].max(), y_pred[:, 1].max())]
    ax2.plot(lims2, lims2, 'r--', lw=2, label='Perfect prediction')
    ax2.set_title("Path 2 (Reflection): Actual vs Predicted")
    ax2.set_xlabel("Actual Range (m)")
    ax2.set_ylabel("Predicted Range (m)")
    ax2.legend()
    # add metrics box
    ax2.text(0.04, 0.97, f"RMSE : {rmse2:.4f} m\nMAE  : {mae2:.4f} m\nR²     : {r2_2:.4f}",
             transform=ax2.transAxes, fontsize=10, verticalalignment='top', fontfamily='monospace',
             bbox=dict(boxstyle='round,pad=0.5', facecolor='lightyellow', edgecolor='#CCCCCC', alpha=0.9))

    plt.tight_layout()
    plt.savefig(plot_dir / "(v2)regression_two_paths.png", dpi=150)
    plt.close()
    print("  [Plot saved] regression_two_paths.png")
```

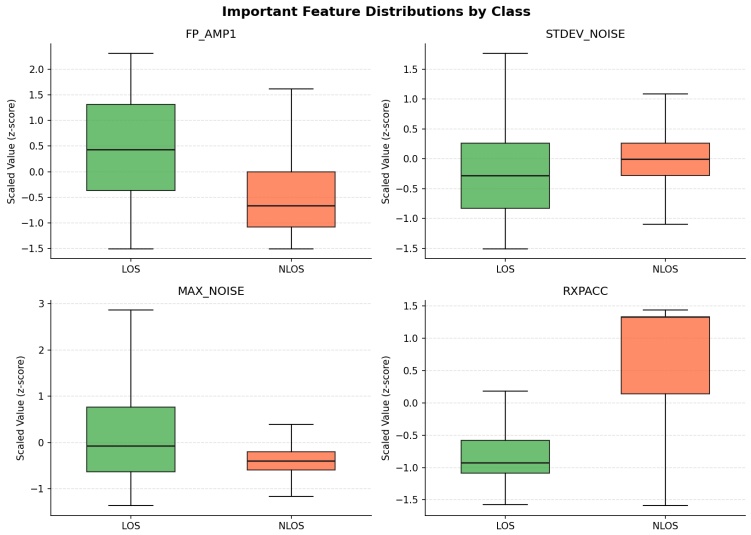
```python
def plot_residuals(y_test, y_pred, plot_dir):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

    for ax, col, color, label in [
        (ax1, 0, 'blue',  'Path 1 (Direct)'),
        (ax2, 1, 'green', 'Path 2 (Reflection)')
    ]:
        residuals = y_test[:, col] - y_pred[:, col]
        rmse = np.sqrt(mean_squared_error(y_test[:, col], y_pred[:, col]))
        mae = mean_absolute_error(y_test[:, col], y_pred[:, col])
        ax.scatter(y_pred[:, col], residuals, alpha=0.3, color=color, s=5)
        ax.axhline(0, color='red', linestyle='--', lw=2)
        ax.set_title(f"Residuals: {label}")
        ax.set_xlabel("Predicted Range (m)")
        ax.set_ylabel("Residual (m)")
        # add metrics box
        ax.text(0.04, 0.97, f"RMSE : {rmse:.4f} m\nMAE  : {mae:.4f} m",    # ADD
                transform=ax.transAxes, fontsize=10, verticalalignment='top', fontfamily='monospace',
                bbox=dict(boxstyle='round,pad=0.5', facecolor='lightyellow', edgecolor='#CCCCCC', alpha=0.9))

    plt.tight_layout()
    plt.savefig(plot_dir / "(v2)residuals_two_paths.png", dpi=150)
    plt.close()
    print("  [Plot saved] residuals_two_paths.png")
```

```python
def main():
    # ── Paths ──────────────────────────────────────────────────────
    root      = Path(__file__).resolve().parent.parent
    data_dir  = root / "data_prep_output"
    plot_dir  = root / "training" / "plots"
    model_dir = root / "training" / "models"
    plot_dir.mkdir(parents=True, exist_ok=True)
    model_dir.mkdir(parents=True, exist_ok=True)

    # ── Load data ──────────────────────────────────────────────────
    try:
        X_train      = np.load(data_dir / "X_train.npy")
        X_test       = np.load(data_dir / "X_test.npy")
        y_train_reg  = np.load(data_dir / "y_train_reg.npy")   # shape (N, 2)
        y_test_reg   = np.load(data_dir / "y_test_reg.npy")    # shape (N, 2)
    except FileNotFoundError as e:
        print(f"Error: Missing file → {e.filename}")
        return

    print(f"X_train : {X_train.shape}  |  y_train_reg : {y_train_reg.shape}")
    print(f"X_test  : {X_test.shape}   |  y_test_reg  : {y_test_reg.shape}")
```

```python
def main():

    # ── Train ──────────────────────────────────────────────────────
    print("\n--- Training Multi-Output Regressor (Path 1 & Path 2) ---")
    # MultiOutputRegressor trains one RF per output column (Path 1 and Path 2)
    # Start with n_estimators=100 to verify it runs, then increase to 200
    base_rf = RandomForestRegressor(
        n_estimators=100,    # increase to 200 for final submission
        max_depth=15,
        min_samples_leaf=5,
        random_state=42,
        n_jobs=-1,
        verbose=1
    )
    reg = MultiOutputRegressor(base_rf)
    reg.fit(X_train, y_train_reg)

    # ── Evaluate ───────────────────────────────────────────────────
    print("\n" + "=" * 50)
    print("      TERMINAL REPORT: TWO-PATH REGRESSION")
    print("=" * 50)

    y_pred_train: np.ndarray = np.asarray(reg.predict(X_train))
    y_pred_test: np.ndarray  = np.asarray(reg.predict(X_test))

    paths = ["Path 1 (Direct)", "Path 2 (Reflection)"]

    print("\n  [Train Results]")
    for i, name in enumerate(paths):
        evaluate(y_train_reg[:, i], y_pred_train[:, i], name)

    print("\n  [Test Results (Unseen Data)]")
    test_rmses = []
    for i, name in enumerate(paths):
        rmse, _, _ = evaluate(y_test_reg[:, i], y_pred_test[:, i], name)
        test_rmses.append(rmse)

    print(f"\n  Average Test RMSE across both paths: {np.mean(test_rmses):.4f} m")

    # ── Plots ──────────────────────────────────────────────────────
    print("\n--- Generating Visualization Files ---")
    plot_two_paths(y_test_reg, y_pred_test, plot_dir)
    plot_residuals(y_test_reg, y_pred_test, plot_dir)

    # ── Save model ─────────────────────────────────────────────────
    model_path = model_dir / "rf_multioutput_regressor.joblib"
    joblib.dump(reg, model_path)
    print(f"\n[Saved] Model → {model_path}")

    # ── Save predictions ───────────────────────────────────────────
    np.save(data_dir / "y_test_pred_reg.npy", y_pred_test)
    print("[Saved] Predictions → y_test_pred_reg.npy")

    print("\n[PROCESS COMPLETE]")


if __name__ == "__main__":
    main()
```

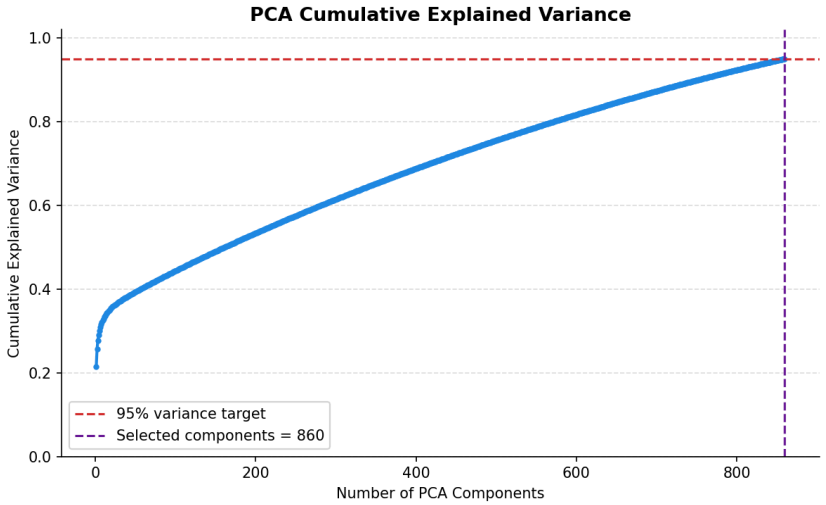
Plots and Results

1. Feature Distributed by Class



The feature distribution boxplot compares the scaled values of FP_AMP1, STDEV_NOISE, MAX_NOISE, and RXPACC across LOS and NLOS classes. FP_AMP1 is notably higher in LOS conditions, reflecting stronger direct-path signals, while RXPACC shows greater spread in NLOS, indicating more variable preamble accumulation due to multipath interference. These distributions confirmed that the engineered and radio features carry meaningful discriminative information for classification

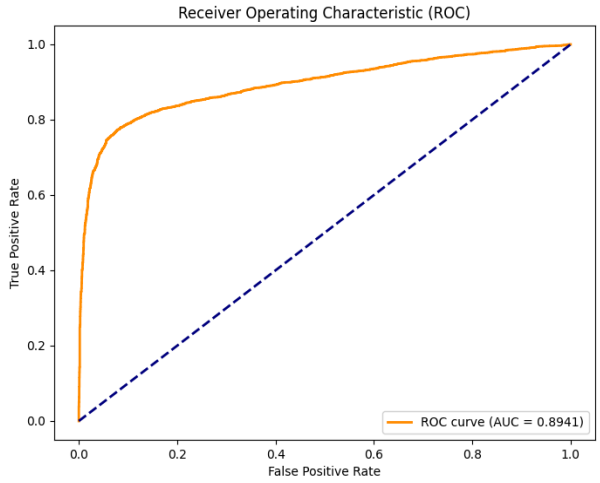
2. PCA Cumulative Explained Variance



The PCA cumulative explained variance plot shows that 860 principal components are required to retain 95% of the total variance from the original 1,017 CIR features. The curve rises steeply in the first 100 components before gradually faltering,

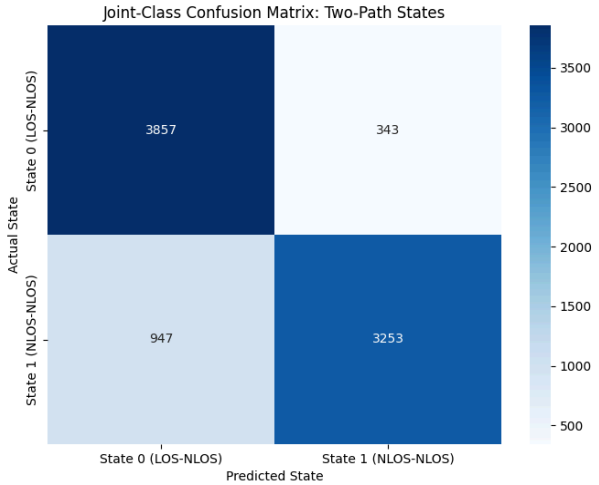
suggesting that while a small subset of components captures the dominant signal patterns, a large number of components are needed to account for the remaining variance. This justifies the relatively high dimensionality of the final feature set.

3. Classifier: ROC Curve, Confusion Matrix and Feature Importance



Predictive Accuracy & Reliability

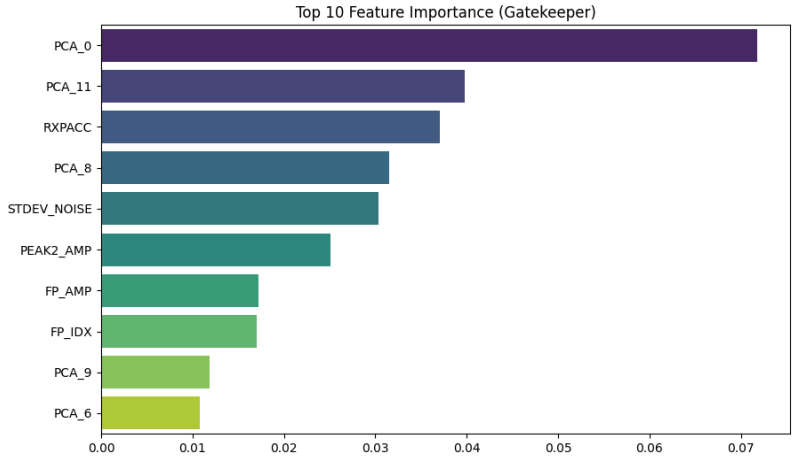
- **Classification Accuracy (84.64%):** The model correctly identifies the signal state in approximately 85% of all cases.
- **ROC AUC (0.8941):** An AUC near 0.90 indicates a highly effective model that can successfully rank the probability of a signal being in State 1 versus State 0.
- **Balanced Performance:** The macro and weighted averages (0.85) confirm that the model performs consistently across both classes, thanks to the balanced class weighting used during training.



Confusion Matrix Analysis (Critical Weakness)

The confusion matrix reveals how the model handles specific signal environments:

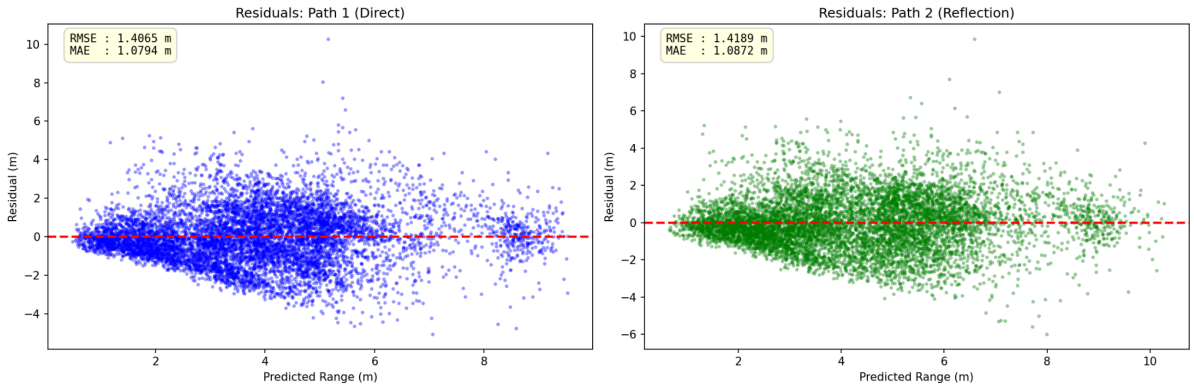
- **Successes:** The model correctly identified **3,857** State 0 (LOS-NLOS) instances and **3,253** State 1 (NLOS-NLOS) instances.
- **False LOS (State 1 predicted as State 0):** There were **947** instances where the model failed to detect an obstruction. This is a "Critical Weakness" because treating an obstructed signal as a clear path leads to significant distance estimation errors.
- **False NLOS (State 0 predicted as State 1):** Only **343** clear signals were mistakenly flagged as obstructed, showing the model is conservative in its "clear path" estimations.



Feature Importance (The Decision Drivers)
The ranking identifies which physical and mathematical attributes most influence the classification:

- **PCA Components (PCA_Comp_0, 11, 8):** These represent compressed signal patterns and hold the highest predictive power, suggesting that the "shape" of the signal wave is more telling than any single metric.
- **RXPACC (Rank 3):** Received Preamble Accumulation is the most critical raw hardware metric, as it directly reflects signal quality.
- **Signal Noise & Peaks:** Features like *STDEV_NOISE* and *PEAK2_AMP* are vital for detecting multi-path interference, where secondary reflections (peaks) often indicate an obstructed primary path.

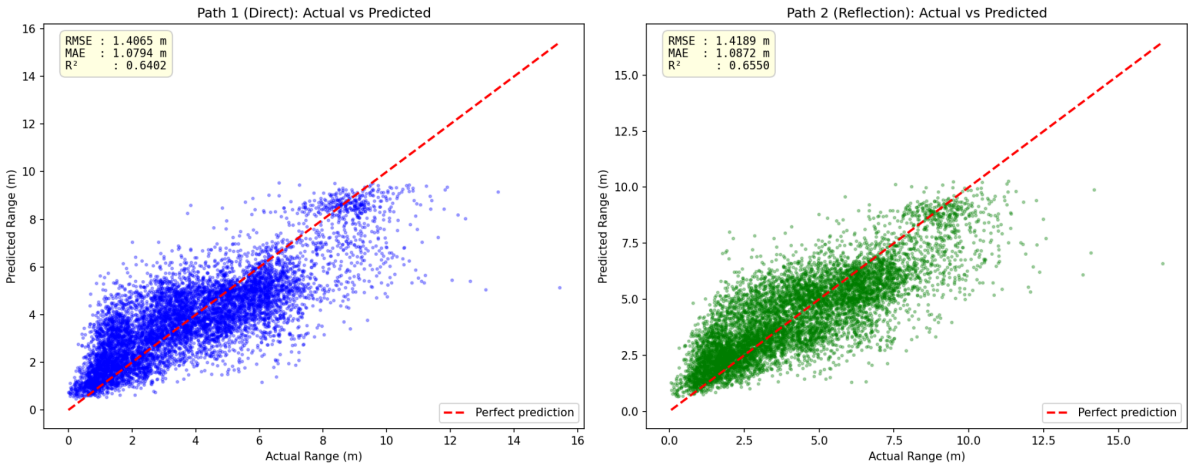
4. Regressor: Residual Plots



The residual plots illustrate the distribution of prediction errors for both direct (LOS) and reflected (NLOS) signal paths. Ideally, residuals should be randomly distributed around zero with constant variance. However, the plots show a funnel-shaped pattern, where the spread of residuals increases with predicted distance. This means that the model's error grows as distance increases. This behaviour is expected in UWB systems due to signal attenuation and stronger multipath effects at longer ranges.

Additionally, the residuals are generally centered around zero, suggesting that the model does not exhibit strong systematic bias, although slight underestimation and overestimation trends are observed in certain ranges. The presence of outliers indicates that the model occasionally fails under severe NLOS conditions.

5. Regressor: Actual vs Predicted Paths



The actual-versus-predicted plots further confirm these findings, showing a strong linear relationship between predicted and actual values, but with increasing dispersion at greater distances. Notably, the reflected (NLOS) path demonstrates slightly better clustering and higher R^2 values compared to the direct path, indicating that the model is able to extract useful information from multipath signals.

We can improve robustness by collecting or weighting more long-range samples so the model does not become biased toward shorter distances.

Overall, while the model captures the general trend effectively, its performance degrades at longer distances and in complex propagation conditions, highlighting the need for improved robustness in real-world warehouse environments.

6. Overall Conclusion

The Hybrid two-path approach demonstrates that both the classifier and regressor can extract meaningful information from UWB CIR signals in mixed LOS/NLOS warehouse environments. The classifier achieves 84.64% accuracy with an AUC of 0.8941, while the regressor estimates direct and reflected path distance with test RMSEs of 1.41m and 1.42m respectively. Performance degrades at longer distances, suggesting that future work should focus on collecting more longer-range samples and exploring distance-weighted training to improve robustness across the full operating range of the warehouse.

Assumptions

The dataset is assumed to be representative of real-world warehouse environments, as it consists of 42,000 balanced samples with an equal split of 21,00 LOS and 21,00 NLOS observations. This balance is treated as a given property of the dataset rather than an artificially imposed condition, and it directly supports the use of balanced class weighting in the classifier.

Although outliers were identified using the IQR rule during preprocessing, they were deliberately not removed or capped. This decision reflects the assumption that extreme signal values in IVB data are likely caused by genuine physical phenomena such as severe multipath interference or signal obstruction, rather than measurement errors. Removing them could therefore discard meaningful NLOS information.

Similarly, no imputation was performed for missing values. The preprocessing stage audits data quality and reports missing entries, but the assumption is that the dataset is sufficiently complete for training without introducing synthetic values that could distort the signal characteristics.

CIR signals are assumed to be approximately stationary within each sample window, meaning the multipath structure does not change significantly during the measurement period. This underpins the validity of applying peak detection and PCA to individual CIR snapshots, Additionally, PCA is fitted exclusively on training data and applied to the test set, ensuring no information leakage between splits.

The warehouse environment used during data collection is assumed to be sufficiently representative of general warehouse layout. That is, the density of metal shelving, the

placement of UWB anchors, and the range of robot movement paths are assumed to reflect conditions that the deployed system would encounter in practice.